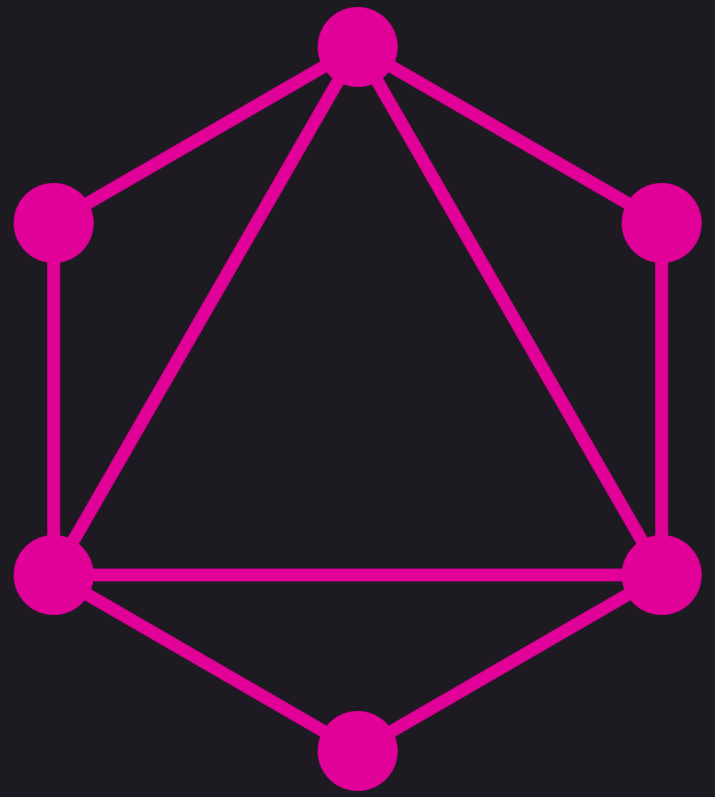
GRAPH QUERY LANGUAGE

GraphQL

Da origem no Facebook a uma API funcionando com Node.js e Apollo Server



O QUE É

Seu domínio de negócio modelado como um grafo

Schema

Você define os tipos de nós e como eles se conectam ou se relacionam entre si.

No cliente

Um padrão parecido com Orientação a Objetos: tipos que fazem referência a outros tipos.

No servidor

O GraphQL define só a interface, então funciona com qualquer backend, novo ou legado.

A CONCEPÇÃO

2012: a reescrita do app do Facebook para iOS

O Facebook abandonava a abordagem de **HTML5 com containers nativos** para focar em **telas nativas**.

As requisições usavam **REST**, e os engenheiros encontraram gargalos que afetavam a performance do aplicativo.



LINHA DO TEMPO

De ferramenta interna a padrão aberto

2012

Uso interno

Conceito e especificação desenvolvidos para alimentar os apps mobile do Facebook.

2015

Código aberto

Com o potencial reconhecido, o Facebook abre o código para a comunidade.

2018

GraphQL Foundation

Projeto transferido para a fundação, tutelada pela Linux Foundation: governança neutra e aberta.

Equipe liderada por **Lee Byron, Nick Schrock e Dan Schafer**

OS PROBLEMAS DO REST

Três gargalos que travavam o app e a equipe

Over-fetching

Respostas rígidas e padronizadas: para mostrar só o nome do usuário, vinha um objeto gigante com dados que não seriam usados.

Under-fetching

O contrário: nome do usuário com seus posts recentes exigia mais de um endpoint. Mais requisições, mais tempo.

Acoplamento

Cada nova combinação de dados numa tela pedia um endpoint novo ou alterado. Um gargalo no ciclo de desenvolvimento.

A SOLUÇÃO

Antes: ”O servidor decide o que enviar”

”O cliente pede o que precisa”

CARACTERÍSTICAS PRINCIPAIS

Um endpoint, consultas declarativas

Uma única rota

`/users`

`/orders`

`/product`



No lugar de várias URLs, um único endpoint.

Consultas declarativas

O cliente envia um payload descrevendo a **árvore de dados** que quer; o servidor responde exatamente com essas informações, nesse formato.

COMO USAR

Defina os tipos e escreva um resolver para cada campo

schema

```
type Query {
  me: User
}

type User {
  name: String
}
```

resolvers

```
// Resolver do campo "me" no tipo "Query"
function resolveQueryMe(_parent, _args, context, _info) {
  return context.request.auth.user;
}

// Resolver do campo "name" no tipo "User"
function resolveUserName(user, _args, context, _info) {
  return context.db.getUserFullName(user.id);
}
```

A resposta tem o mesmo formato da pergunta

query

```
{
  me {
    name
  }
}
```



resposta

```
{
  "data": {
    "me": {
      "name": "Ettore Saburi"
    }
  }
}
```

NA PRÁTICA

Criando a API de um blog com Node.js e Apollo Server

terminal

```
mkdir graphql-demo && cd graphql-demo
npm init -y
npm pkg set type=module
npm install @apollo/server graphql
```

O index.js em 4 partes

1. "Banco de dados" em memória
2. Schema (SDL)
3. Resolvers: uma função por campo
4. Subir o servidor na porta 4000

NA PRÁTICA · SCHEMA

Tipos, enum, input, Query e Mutation

```
enum Role { ADMIN AUTHOR READER }

type User {
  id: ID!
  name: String!
  email: String!
  role: Role!
  posts: [Post!]!
}

type Post {
  id: ID!
  title: String!
  content: String
  author: User!
}
```

```
input CreatePostInput {
  title: String!
  content: String
  authorId: ID!
}

type Query {
  users: [User!]!
  user(id: ID!): User
  posts(limit: Int = 10): [Post!]!
}

type Mutation {
  createPost(input: CreatePostInput!): Post!
  deletePost(id: ID!): Boolean!
}
```

Uma função para cada campo

```
Query: {
  users: () => users,
  user: (_parent, args) =>
    users.find((u) => u.id === args.id),
  posts: (_parent, args) =>
    posts.slice(0, args.limit),
},
// Resolvers de relacionamento
User: {
  posts: (user) =>
    posts.filter((p) => p.authorId === user.id),
},
Post: {
  author: (post) =>
    users.find((u) => u.id === post.authorId),
},
```

```
// Sobe o servidor
const server = new ApolloServer({
  typeDefs, resolvers
});
const { url } = await startStandaloneServer(
  server, { listen: { port: 4000 } }
);
console.log(`Servidor pronto em ${url}`);
```

TESTANDO NO APOLLO SANDBOX

Usuário e seus posts em uma única requisição

query

```
{
  user(id: "1") {
    name
    posts {
      title
    }
  }
}
```

resposta

```
{
  "data": {
    "user": {
      "name": "Ettore Saburi",
      "posts": [
        { "title": "Introdução ao GraphQL" },
        { "title": "GraphQL vs REST" }
      ]
    }
  }
}
```

Com REST seriam **duas requisições**: /users/1 e /users/1/posts.

ESCREVENDO DADOS

Mutations criam, alteram e apagam

mutation

```
mutation {
  createPost(input: {
    title: "Meu primeiro post",
    content: "GraphQL é legal!",
    authorId: "1"
  }) {
    id
    title
    author { name }
  }
}
```

resposta

```
{
  "data": {
    "createPost": {
      "id": "3",
      "title": "Meu primeiro post",
      "author": {
        "name": "Ettore Saburi"
      }
    }
  }
}
```

SEGURANÇA DE TIPOS

O schema valida a consulta antes de executar

Se a query pede um campo que não existe no schema, como **telefone**, o servidor recusa antes mesmo de rodar os resolvers.

erro

```
Cannot query field "telefone" on
type "User".
```

RESUMINDO

Por que usar GraphQL

Sem desperdício

Nem over-fetching nem under-fetching: o cliente pede exatamente o que precisa.

Um endpoint

Uma rota só, com consultas declarativas sobre um grafo de tipos.

Contrato tipado

O schema documenta a API e valida cada consulta.

Qualquer backend

Só define a interface: funciona com sistemas novos ou legados.